

GESTÃO E QUALIDADE DE SOFTWARE

SPRINT 2 — RELATÓRIO DE EVOLUÇÃO

1. Identificação

Nome do Projeto	E-Commerce Checkout Engine
URL do Repositório	https://github.com/MPicolli/ecommerce-checkout-engine
Integrantes	Eric Levi Sena Silveira (RA 10723114930) José Augusto Ferreira (RA 10725220369) Leonardo de Medeiros Binatti (RA 1072322655) Matheus Picolli Ishibashi (RA 10724262102) Vithor Massing dos Santos (RA 1072316868)
Data de Entrega	20/05/2026

2. Funcionalidades Entregues

#	Funcionalidade	Status	Testada automaticamente ?
1	Modelagem do domínio (Produto, Cliente, Carrinho, ItemCarrinho, Cupom, ResumoPedido, TipoCupom)	☒ Completa ☐ Parcial	☒ Sim ☐ Não
2	Exceções customizadas de negócio (CarrinhoVazioException, CupomInvalidoException, PontosInsuficientesException)	☒ Completa ☐ Parcial	☒ Sim ☐ Não
3	Motor de cálculo de frete dinâmico por CEP e peso (FreteService)	☒ Completa ☐ Parcial	☒ Sim ☐ Não
4	Serviço de validação e aplicação de cupons (CupomService)	☒ Completa ☐ Parcial	☒ Sim ☐ Não
5	Serviço de pontos de fidelidade (FidelidadeService)	☒ Completa ☐ Parcial	☒ Sim ☐ Não

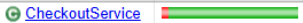
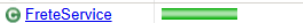
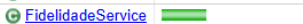
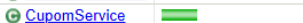
3. Testes Automatizados

3.1 Resumo da cobertura

Para a Sprint 2, foram desenvolvidos 36 testes automatizados de unidade focados nas regras de negócio centrais do motor de checkout. Os testes cobrem integralmente os fluxos de cálculo de frete, aplicação de cupons de desconto, regras do programa de fidelidade e o processamento final do checkout do e-commerce. O relatório gerado pela ferramenta JaCoCo aponta que atingimos uma

cobertura geral de 97% das instruções (linhas de código) do sistema, com 83% de cobertura de ramificações lógicas (complexidade de caminhos).

com.unisul.ecommerce.service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
CheckoutService		94%		62%	9	17	3	61	0	5	0	1
FreteService		100%		93%	2	19	0	26	0	4	0	1
FidelidadeService		100%		100%	0	13	0	23	0	6	0	1
CupomService		100%		80%	2	8	0	15	0	3	0	1
Total	13 of 536	97%	13 of 78	83%	13	57	3	125	0	18	0	4

Total de testes escritos	Cobertura reportada pela ferramenta	Framework utilizado
36 testes (todos com sucesso)	Média geral do projeto: 97% Instructions 83% Branches	JUnit 5 (Jupiter) + Mockito

3.2 Exemplos de testes

Descreva 2 testes que o grupo considera mais importantes

Teste 1 — Validação do frete grátis no valor limite (FreteServiceTest)

Verifica se compras com valor total exatamente igual a **R\$ 200,00** recebem frete gratuito conforme a RN05. Esse cenário é importante porque testa um limite exato da regra de negócio. Caso falhe, clientes poderiam receber cobrança incorreta de frete.

Teste 2 — Finalização completa do pedido com serviços simulados (CheckoutServiceTest)

Verifica o processamento do checkout utilizando Mockito para simular dependências como FreteService, CupomService e FidelidadeService. Esse teste valida a consolidação correta dos cálculos e a geração do ResumoPedido. Caso falhe, o fluxo principal de compra pode gerar resultados incorretos.

3.3 O que ainda não está coberto por testes

Quais funcionalidades ou cenários importantes ainda não têm teste automatizado?

Alguns cenários planejados ainda não possuem cobertura automatizada completa, principalmente casos adicionais de exceção e análise de valor limite definidos na documentação. Entre eles estão cenários complementares do CupomService (valor mínimo de compra e cupons adicionais), além de testes integrados do fluxo completo do checkout envolvendo a interação entre frete, cupons e fidelidade sem uso de mocks. Também permanecem como melhoria futura testes end-to-end para validação do comportamento completo do sistema.

4. Controle de Versão e Boas Práticas

4.1 Evidências de uso do Git

Total de commits no período	52 commits (18/03/2026 a 20/05/2026) — Histórico completo disponível no repositório oficial do grupo: https://github.com/MPicolli/ecommerce-checkout-engine/commits/main
Branches utilizadas	- main (Branch principal estável) - docs/documentacao-tecnica - chore/configura-jacoco (Configuração da infraestrutura de testes) - feature/checkout-service - feature/cupom-service - feature/fidelidade-service - feature/teste-fidelidade-service
Pull Requests / revisões	9 pull requests (18/03/2026 a 20/05/2026) — Foram gerenciados os Pull Requests para a integração das principais features e documentações da Sprint 2. O processo contou com a atribuição de revisores (<i>Reviewers</i>) entre os membros da equipe para garantir a validação prévia do código e a aprovação formal antes do merge na branch main.
CI configurado?	☐ Sim (descreva a ferramenta) ☒ Não ainda

4.2 Reflexão sobre o processo

O que funcionou bem no trabalho em equipe? O que foi difícil?

O grupo adotou bem o fluxo de trabalho com branches e Pull Requests desde o início, o que facilitou a organização e reduziu conflitos de código. A divisão de responsabilidades foi clara, permitindo que cada integrante focasse em uma funcionalidade específica. O padrão Conventional Commits também foi seguido na maior parte do desenvolvimento, tornando o histórico do repositório mais organizado e legível.

O principal desafio foi sincronizar o ritmo de desenvolvimento entre os integrantes, já que algumas funcionalidades dependiam da conclusão de outras branches já integradas. Outro ponto identificado foi que as revisões de código ocorreram de forma parcial em alguns momentos. Apesar de o fluxo definido pelo grupo prever revisões por outros membros antes do merge, houve casos em que Pull Requests foram integrados sem seguir completamente esse processo. Isso representa um uso incompleto das práticas estabelecidas no GitHub Flow e foi identificado pelo grupo como um ponto de melhoria para a Sprint 3. Durante a integração também surgiram ajustes posteriores relacionados à cobertura de testes, padronização de cálculos monetários com `BigDecimal` e correções pontuais de implementação.

5. Ajustes de Escopo

O escopo original do pré-projeto mudou? Se sim, o que mudou e por quê?

O escopo original definido na Sprint 1 foi mantido. As cinco funcionalidades previstas (modelagem do domínio, motor de frete, validação de cupons, gestão de fidelidade e processamento de checkout) permaneceram no planejamento e foram desenvolvidas ao longo da Sprint 2.

Durante a implementação, foi necessário revisar e detalhar algumas regras de negócio para reduzir ambiguidades e alinhar a documentação ao comportamento real do sistema. Foram incluídos ajustes relacionados ao limite percentual de cupons, validação de CEP, tratamento de exceções em cenários de fidelidade e regras de programação defensiva nas entidades.

Também foi adotada uma estratégia incremental de desenvolvimento, com serviços implementados de forma independente. Essa abordagem permitiu que os integrantes trabalhassem em paralelo, reduzindo bloqueios entre funcionalidades e facilitando a integração posterior no CheckoutService.

6. Plano para a Sprint 3

#	O que falta fazer	Responsável(is)
1	Finalizar testes pendentes e complementar cenários de exceção e análise de valor limite	José, Leonardo, Matheus
2	Revisar documentação técnica (README, cenários de testes e especificação) para alinhamento com a implementação final	Eric, Vithor
3	Reforçar o processo de revisão obrigatória de Pull Requests conforme o fluxo definido no projeto	Todos
4	Preparar demonstração final e alinhar a apresentação das decisões técnicas adotadas	Todos
5	Avaliar a implementação de uma interface mínima em console para facilitar a demonstração do fluxo completo do checkout	Todos

Lembre: o professor vai olhar o histórico de commits — o trabalho precisa aparecer no repositório, não só neste documento.